



Fundamentos de Inteligencia Artificial y Agentes Inteligentes

INTELIGENCIA ARTIFICIAL I · IS-484 · 2026-II

Escuela Profesional de Ingeniería de Sistemas

Ing. Leidy Rosmery Maldonado Chauca

1. Objetivos

- Clasificar sistemas de Inteligencia Artificial según el tipo de problema que resuelven y el área de aplicación a la que pertenecen.
- Representar un sistema de IA mediante el esquema formal de agente inteligente (PEAS), determinando su racionalidad y si corresponde a IA estrecha o IA General.
- Configurar un entorno de trabajo en Python mediante Jupyter Notebook, con las bibliotecas oficiales del curso: NumPy, Pandas, Matplotlib, Scikit-learn y SciPy.
- Implementar un agente reactivo simple en Python como primera conexión entre el concepto de agente y su representación computacional.

2. Equipos, Herramientas o Materiales

- Computadora con Windows 10 o Windows 11
- Acceso a internet
- Python 3.x
- Jupyter Notebook
- Editor de código: Visual Studio Code (recomendado) o Jupyter Notebook clásico

3. Referencia para el Laboratorio

3.1. IA estrecha frente a IA General

IA estrecha: un sistema diseñado para resolver una sola tarea específica (por ejemplo, solo reconocer rostros, o solo recomendar películas). No puede hacer nada distinto a esa tarea sin ser rediseñado.

IA General (AGI): un sistema capaz de razonar y resolver cualquier tipo de tarea, igual que una persona. Todavía no existe ningún sistema así funcionando en el mundo real: es un tema de investigación, no algo que se pueda usar hoy.

Todo lo que se analiza y construye en este laboratorio es **IA estrecha**.

3.2. Esquema PEAS del agente inteligente

Un **agente** percibe su entorno mediante sensores y actúa sobre él mediante actuadores. Es **racional** cuando, dada su secuencia de percepciones, elige la acción que maximiza su medida de desempeño esperada. El esquema PEAS describe cualquier agente mediante estos cuatro elementos:

- **P: Performance (Desempeño):** ¿cómo se mide si el agente lo está haciendo bien?
- **E: Environment (Entorno):** ¿en qué contexto opera?
- **A: Actuators (Actuadores):** ¿qué acciones puede ejecutar el agente?
- **S: Sensors (Sensores):** ¿qué percibe el agente del entorno?

3.3. Tipos de agente

- **Reactivo simple:** actúa solo según la percepción actual, sin ningún tipo de memoria del pasado. Por ejemplo, un termostato enciende o apaga la calefacción únicamente según la temperatura que mide en ese instante, sin recordar mediciones anteriores.
- **Basado en modelos:** mantiene un estado interno que representa aspectos del entorno que no puede observar directamente en el momento. Un robot aspirador que recuerda qué zonas de la casa ya limpió, aunque no las esté viendo, es un ejemplo de este tipo.
- **Basado en objetivos:** evalúa distintas secuencias de acciones posibles y elige la que lo acerca a una meta explícita. Un sistema de navegación GPS que planifica una ruta completa hacia un destino, en lugar de decidir cuadra por cuadra, actúa con esta lógica.
- **Basado en utilidad:** en lugar de un objetivo binario (lo logra o no lo logra), compara varias acciones posibles según qué tan buenas son, no solo si funcionan. Un sistema de recomendación de rutas que no solo busca llegar, sino la opción más rápida, económica y segura al mismo tiempo, usa una función de utilidad.
- **De aprendizaje:** mejora su regla de decisión con la experiencia, ajustando su comportamiento a partir de datos en lugar de seguir reglas fijas escritas de antemano. Un filtro de spam que se vuelve más preciso mientras más correos marca el usuario como spam o no spam es un agente de este tipo.

3.4. Áreas de aplicación de la IA (taxonomía oficial)

Las áreas de aplicación de la Inteligencia Artificial son las siguientes:

Área	Qué resuelve	Ejemplo cotidiano
Visión artificial	Que una máquina reconozca objetos, rostros o escenas en imágenes o video	Cámaras de seguridad que detectan personas, reconocimiento facial de un celular
Procesamiento de lenguaje natural (NLP)	Que una máquina entienda o genere texto o habla humana	Traductores automáticos, chatbots, corrección ortográfica
Sistemas de recomendación	Predecir qué le gustaría a un usuario según su comportamiento previo	“Porque viste esto” en Netflix, productos relacionados en una tienda en línea

Área	Qué resuelve	Ejemplo cotidiano
Robótica	Que un sistema físico perciba su entorno y actúe sobre él en el mundo real	Aspiradoras robot, brazos robóticos de ensamblaje, drones autónomos
Sistemas expertos	Reglas de conocimiento codificadas de un experto humano para diagnosticar o decidir	Sistemas de diagnóstico médico basados en reglas
Reconocimiento de patrones	Identificar regularidades o similitudes en datos	Detección de huellas dactilares, identificación de anomalías en señales
Análisis predictivo	Estimar valores o eventos futuros a partir de datos históricos	Predicción de tráfico, pronóstico de demanda

Nota: Estas áreas no son excluyentes entre sí: un mismo sistema puede tocar varias a la vez (ej. un asistente de voz usa NLP y análisis predictivo). Para la Actividad 4.3, identificar el área **predominante**: la que mejor describe el problema central que resuelve el sistema.

4. Actividades de Laboratorio

4.1. Instalación y verificación del entorno de trabajo

Parte A: Diagnóstico rápido

Indicar si ya se cuenta con Python instalado y funcional en el equipo. Si es así, continuar directamente en la Parte D.

Parte B: Verificar la instalación de Python

```
python --version
```

Si el comando no es reconocido, descargar e instalar Python desde <https://www.python.org/downloads/>.

⊗ **Atención:** En Windows, durante la instalación es obligatorio marcar la casilla “Add Python to PATH” en la primera pantalla del instalador. Omitir este paso es la causa más frecuente de que el comando `python` no sea reconocido después en la terminal.

Parte C: Crear un entorno virtual e instalar Jupyter

```
python -m venv venv_ia1
.\venv_ia1\Scripts\Activate.ps1
```

```
pip install numpy pandas matplotlib scikit-learn scipy jupyter
```

Nota: Si PowerShell muestra un error de “ejecución de scripts deshabilitada” (PSSecurityException) al activar el entorno, ejecutar una sola vez `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` y confirmar con `S`. No requiere permisos de administrador.

Parte D: Verificar que Jupyter se ejecuta correctamente

```
jupyter notebook
```

Debe abrirse una pestaña del navegador con el panel de Jupyter. Crear un nuevo notebook (**New** → **Python 3**) y renombrarlo como IS484_Lab01_Apellido.ipynb.

Nota: Si el equipo del laboratorio no permite instalar software (sin permisos de administrador), usar Google Colab (<https://colab.research.google.com>) como alternativa temporal.

4.2. Verificación de las bibliotecas del curso

Parte A: Verificar versiones

```
import numpy as np
import pandas as pd
import matplotlib
import sklearn
import scipy

print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Scikit-learn:", sklearn.__version__)
print("SciPy:", scipy.__version__)
```

Parte B: Verificación funcional (ejemplo: un sistema anti-fraude)

Se genera un conjunto simulado de 300 transacciones (con más probabilidad de fraude en horario de madrugada y en montos altos) y se visualiza, para observar el patrón antes de aplicar cualquier algoritmo (lo que se hará desde la Semana 5):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
n = 300

hora = np.random.randint(0, 24, n)
monto = np.random.exponential(scale=150, size=n) + 10

# Fraude mas probable de madrugada (0-5h, 23h) y en montos altos
prob_fraude = 0.03 + 0.35 * ((hora <= 5) | (hora >= 23)) + 0.25 * (monto > 400)
prob_fraude = np.clip(prob_fraude, 0, 0.9)
es_fraude = (np.random.rand(n) < prob_fraude).astype(int)

transacciones = pd.DataFrame({"hora": hora, "monto": monto, "es_fraude": es_fraude})

colores = {0: "#0ca30c", 1: "#d03b3b"} # verde = legitima, rojo = fraude
etiquetas = {0: "Legitima", 1: "Fraude"}

fig, ax = plt.subplots(figsize=(10, 6), facecolor="#f0f0f0")
ax.set_facecolor("#f0f0f0")

for valor in [0, 1]:
    subset = transacciones[transacciones["es_fraude"] == valor]
    ax.scatter(
        subset["hora"], subset["monto"],
```

```

        s=70, alpha=0.75,
        c=colores[valor],
        edgecolors="white", linewidths=0.6,
        label=etiquetas[valor],
    )

ax.set_title("Transacciones por hora y monto", fontsize=15, color="#0b0b0b", pad=14)
ax.set_xlabel("Hora del dia", fontsize=11, color="#52514e")
ax.set_ylabel("Monto (S/)", fontsize=11, color="#52514e")
ax.set_xticks(range(0, 24, 2))
ax.grid(True, color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(title="Tipo de transaccion", frameon=False, loc="upper right")

plt.tight_layout()
plt.show()

print("Total transacciones:", len(transacciones),
      "| Fraudes:", transacciones["es_fraude"].sum())

```

Nota: Si alguna importación falla, revisar la instalación con `pip list` dentro del entorno virtual activo. No avanzar a la Sección 4.3 con un entorno incompleto.

4.3. Clasificación de sistemas con Inteligencia Artificial

Parte A: Elegir dos sistemas de la siguiente lista (o proponer uno propio):

- Recomendador de Netflix/Spotify, filtro anti-spam de Gmail, asistente de voz (Siri/Alexa)
- Sistema de reconocimiento facial, chatbot de atención al cliente, predicción de tráfico
- Sistema anti-fraude bancario en tiempo real, balanceador de carga inteligente, sistema de autenticación biométrica

Parte B: Completar, para cada sistema elegido, entrada, salida, tipo de problema (clasificación/regresión/clustering/generación) y área de IA, usando **únicamente** las siete categorías oficiales listadas en la Sección 3.4.

4.4. Representación de un agente inteligente (PEAS)

Parte A: Retomar uno de los sistemas técnicos analizados en la Sección 4.3 (fraude, balanceo de carga, biometría), para exigir mayor rigor que con un ejemplo de consumo masivo.

Parte B: Completar la ficha PEAS del agente:

- Percepción (S), Acciones (A), Entorno (E), Objetivo, Medida de desempeño (P)
- Tipo de agente (reactivo, basado en modelos, en objetivos, en utilidad, de aprendizaje)
- ¿Es racional? Justificar en 1 a 2 líneas
- ¿Corresponde a IA estrecha o IA General? Justificar

4.5. Entregable del Trabajo en Clase

- Tabla de clasificación de sistemas (Sección 4.3, Parte B).

- Ficha PEAS del sistema técnico elegido (Sección 4.4, Parte B), incluyendo racionalidad e IA estrecha/General.

Nota: Entregar ambas fichas (foto legible o documento escaneado) en la tarea “Trabajo en Clase” del aula virtual, antes de finalizar la sesión.

5. Ejercicio Práctico de Laboratorio

5.1. Planteamiento del problema

Se quiere automatizar el control de temperatura de un ambiente mediante un sensor y un sistema de climatización (aire acondicionado y calefacción). El sistema debe decidir, en cada momento, qué acción tomar según la temperatura medida.

5.2. Diseño del agente (ficha PEAS)

Completar, **antes de escribir código**, la ficha PEAS de este problema:

Percepción (S)

Acciones (A)

Entorno (E)

Objetivo

Medida de
desempeño (P)

5.3. De PEAS a reglas de decisión

Traducir el objetivo anterior en reglas concretas, completando la tabla:

Condición	Acción a tomar
Si la temperatura está muy por encima de la deseada	
Si la temperatura está muy por debajo de la deseada	
Si la temperatura está cerca de la deseada	

5.4. Implementación

Completar el código a partir de **sus propias reglas** de la tabla anterior (reemplazar cada # COMPLETAR):

```
def agente_termostato(temperatura_actual, temperatura_objetivo=22, margen=1.0):
    """
    Agente reactivo simple (sin memoria, sin modelo del entorno).
    Completar el docstring con el propio analisis PEAS de la Seccion 5.2.
    """
    diferencia = temperatura_actual - temperatura_objetivo

    if diferencia > margen:
        return # COMPLETAR: accion si hace mucho calor
    elif diferencia < -margen:
        return # COMPLETAR: accion si hace mucho frio
    else:
        return # COMPLETAR: accion si la temperatura esta bien
```

⊗ **Atención:** Al escribir el código a mano (en vez de copiar y pegar), Python no perdona la sangría: cada línea dentro de `def`, `if`, `elif` o `else` debe llevar 4 espacios más que la línea anterior que termina en dos puntos (:). Jupyter agrega la sangría automáticamente al presionar Enter después de una línea con `:`.

5.5. Simulación

Probar el agente con la siguiente secuencia de percepciones, y agregar **dos valores propios**, incluyendo al menos un caso límite (una temperatura justo en el borde del margen):

```
percepciones = [18.0, 19.5, 21.8, 24.3, 26.0, 23.0, 20.5]

for t in percepciones:
    accion = agente_termostato(t)
    print(f"Percepcion: {t} C -> Accion del agente: {accion}")
```

5.6. Reflexión

Responder en una celda markdown o comentario: ¿este agente termostato es racional? ¿Bajo qué condiciones del entorno dejaría de serlo (por ejemplo, si el sensor de temperatura falla o si el margen es demasiado amplio)?

5.7. Entrega mediante GitHub Classroom

- Aceptar la asignación del laboratorio ingresando al enlace proporcionado por el docente en GitHub Classroom. Esto genera automáticamente un repositorio individual para el estudiante.
- Clonar el repositorio dentro de la terminal:

```
git clone URL_DEL_REPOSITORIO
cd nombre_del_repositorio
```

- Ubicar el notebook o script del ejercicio (Secciones 5.2 a 5.6) dentro del repositorio clonado.
- Registrar y subir los cambios:

```
git add .
git commit -m "Laboratorio 01: fundamentos de IA y agente reactivo"
git push origin main
```

Nota: Si Git no está instalado, descargar el instalador desde <https://git-scm.com/downloads> y ejecutarlo con las opciones por defecto.

Si es la primera vez que se emplea Git en el equipo, configurar la identidad con `git config --global user.name "Nombre Apellido"` y `git config --global user.email "correo@ejemplo.com"`.

5.8. Entregable del Ejercicio de Laboratorio

- Repositorio de GitHub Classroom actualizado, con al menos un *commit* que evidencie el avance del laboratorio.
- Notebook (.ipynb) con las celdas de verificación, la prueba funcional y el agente reactivo completado, ejecutadas sin errores.
- Ficha PEAS del agente termostato (Sección 5.2) y tabla de reglas (Sección 5.3), completadas antes del código.

5.9. Extensión para la casa

El agente termostato decide en función de **una sola** percepción (la temperatura). Esta extensión pide diseñar un agente que decida en función de **dos** percepciones a la vez, combinándolas.

Problema: un sistema de riego automático de jardín debe decidir si regar o no, considerando:

- `humedad_suelo`: un número de 0 (seco) a 100 (saturado)
- `lloverá_hoy`: `True` o `False`, según el pronóstico

Parte A: Completar una ficha PEAS propia para este problema (Percepción, Acciones, Entorno, Objetivo, Medida de desempeño), siguiendo el mismo formato de la Sección 5.2.

Parte B: Proponer las reglas de decisión, considerando que ahora deben combinar dos condiciones con `and` / `or`. Por ejemplo: “si el suelo está seco Y no lloverá hoy, entonces regar”.

Parte C: Escribir la función completa desde cero (sin plantilla esta vez, ya que el proceso se practicó en la Sección 5.4):

```
def agente_riego(humedad_suelo, llovera_hoy):  
    # Escribir aquí la logica completa, basada en las reglas de la Parte B  
    pass
```

Parte D: Probar el agente con al menos 5 combinaciones distintas de `humedad_suelo` y `llovera_hoy`, incluyendo un caso donde ambas condiciones compitan entre sí (por ejemplo, suelo muy seco pero con lluvia pronosticada).

Parte E: Responder: ¿qué tan distinto fue diseñar este agente respecto al termostato? ¿A qué tipo de agente de la Sección 3.3 corresponde?

Nota: Subir esta extensión como un segundo *commit* al mismo repositorio de GitHub Classroom, antes de la Semana 2.

6. Anexo: Solución de Errores Frecuentes

Error / síntoma	Causa	Solución
'python' no se reconoce como un comando	Python no fue agregado al PATH durante la instalación	Reinstalar Python marcando la casilla "Add Python to PATH", o ejecutar el instalador nuevamente y elegir "Modify"
ModuleNotFoundError: No module named 'sklearn'	La biblioteca no se instaló en el entorno activo	Ejecutar <code>pip install scikit-learn</code> dentro del mismo entorno virtual que se está usando
'jupyter' no se reconoce como un comando	Jupyter no se instaló en el entorno activo, o el entorno virtual no está activado	Verificar que el venv esté activado (el nombre debe aparecer entre paréntesis en la terminal) y ejecutar <code>pip install jupyter</code> nuevamente
pip install falla por permisos	Intento de instalación global sin privilegios de administrador	Usar un entorno virtual (venv) en lugar de instalar de forma global, o agregar <code>--user</code> al comando
no se reconoce como nombre de un cmdlet... al activar el venv	Falta la extensión <code>.ps1</code> : <code>activate</code> (sin extensión) es el script para Bash, no para PowerShell	Ejecutar <code>.\venv\ia1\Scripts\Activate.ps1</code> (con <code>Activate.ps1</code> , no solo <code>activate</code>)
...no se puede cargar porque la ejecución de scripts está deshabilitada... (PSSecurityException)	La política de ejecución de PowerShell bloquea scripts <code>.ps1</code> por defecto	Ejecutar una sola vez <code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code> y confirmar con <code>S</code> ; no requiere permisos de administrador
El gráfico de Matplotlib no aparece	Falta <code>plt.show()</code> al final de la celda	Agregar <code>plt.show()</code> después de construir el gráfico
fatal: not a git repository	El comando <code>git</code> se ejecutó fuera de la carpeta clonada	Verificar con <code>cd</code> que se está dentro del repositorio antes de usar <code>git add/commit/push</code>
fatal: repository not found	URL incorrecta o asignación de Classroom no aceptada	Verificar el enlace y confirmar que se aceptó la asignación en GitHub Classroom antes de clonar
"Please tell me who you are"	Identidad de Git no configurada en el equipo	Ejecutar <code>git config --global user.name</code> y <code>user.email</code>